

Data Warehousing & Business Intelligence



Hassan Younas

21I-1788

DS-A

Question 1

We can start by applying indexes on columns used in JOIN and WHERE clauses to speed up data retrieval. The most feasible approach would be via R+ trees. We can Consider creating materialized views for frequently used or resource-intensive queries. Materialized views(although not in SQL workbench) store the result of a query, allowing faster access to the aggregated data. The best approach for using partition would be the vertical partition. We can separate frequently used columns to reduce the total amount of scans required for the joins.

```
SELECT concat(Students.FirstName, ' ', Students.LastName), Department.DepartmentName,  
Course.CourseName, Credits.Credits, Enrollment.EnrollmentDate, Grades.GradeValue  
  
FROM Students JOIN Departments ON Students.DepartmentID = Department.DepartmentID  
  
JOIN Courses ON Department.DepartmentID = Course.DepartmentID  
  
JOIN Enrollments ON Course.CourseID = Enrollments.CourseID  
  
JOIN Grades ON Enrollments.EnrollmentID = Grades.EnrollmentID;
```

This join first starts by joining departments with courses first since only a handful of departments exist and only a handful of courses are offered by the departments. Then we move onto the grading for the enrolled students and finally we get the required information in the way by which lowest amounts of scans of the disk are required and concatenated data forwarded is less.

Question 2

City wise Bitmap:

City	T	V
R1	1	0
R2	0	1
R3	1	0
R4	0	1
R5	1	0
R6	0	1
R7	1	0
R8	0	1

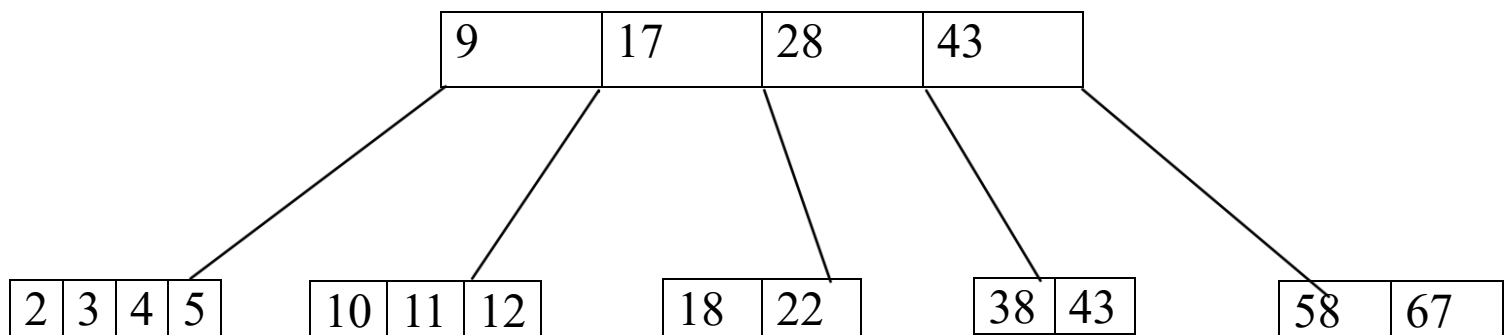
The table consists of columns, each representing a distinct item category: "H" for home entertainment, "C" for computers, "P" for phones, and "S" for security. Each row in the table corresponds to a specific entry of data. A cell with the value "1" indicates the presence of the respective item category for the associated record. Conversely, a cell with the value "0" signifies the absence of the corresponding item category for that record.

Item wise Bitmap:

Item	H	C	P	S
R1	1	0	0	0
R2	0	1	0	0
R3	0	0	1	0
R4	0	0	0	1
R5	1	0	0	0
R6	0	1	0	0
R7	0	0	1	0
R8	0	0	0	1

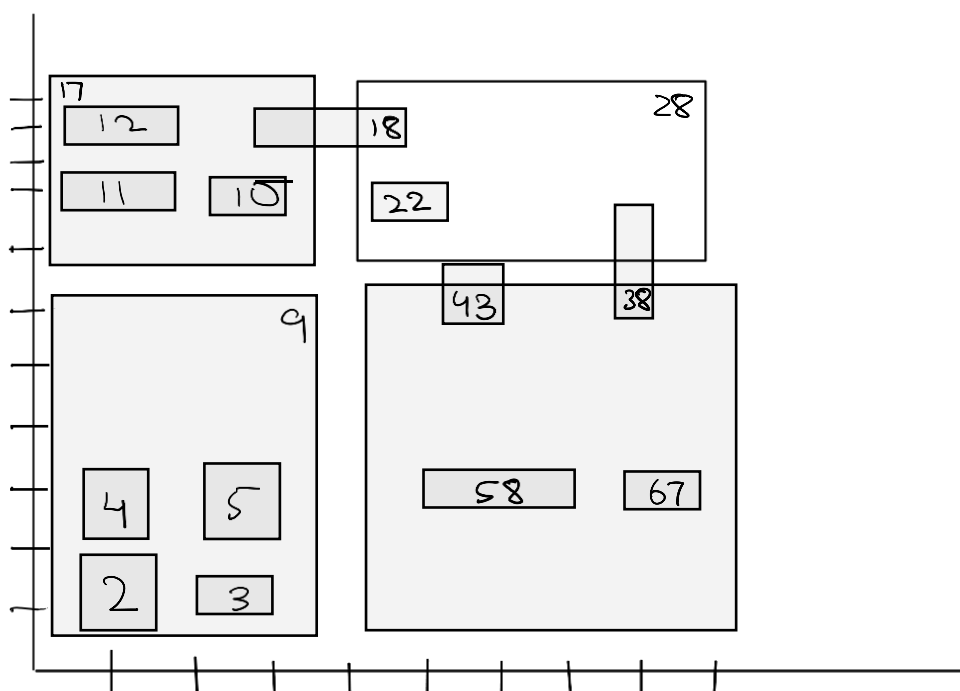
The table consists of columns, each representing a distinct city: "T" for Toronto and "V" for Vancouver. Each row in the table corresponds to a specific entry of data. A cell with the value "1" indicates the presence of the respective city for the associated record. Conversely, a cell with the value "0" indicates the absence of the corresponding city.

Question 3



Part B

MBR=4



Question 4

Task 1:

In my opinion the most frequently used data would consist of:

The total amount, the date of transaction and product id and region.

```
CREATE TABLE FrequentlyUsedData (  
    TransactionID INT,  
    ProductID INT,  
    TransactionDate DATETIME,  
    Region VARCHAR,  
    TotalAmount FLOAT,  
    PRIMARY KEY (TransactionID)  
);  
  
INSERT INTO FrequentlyUsedData (TransactionID, ProductID, TransactionDate, Region, TotalAmount)  
SELECT TransactionID, ProductID, TransactionDate, Region, TotalAmount FROM SalesTransactions;  
  
CREATE TABLE SeldomUsedData (  
    TransactionID INT,  
    CategoryID INT,  
    CustomerID INT,  
    Quantity INT,  
    UnitPrice FLOAT,  
    Discount FLOAT,  
    PRIMARY KEY (TransactionID),  
    FOREIGN KEY (TransactionID) REFERENCES FrequentlyUsedData(TransactionID)  
);  
  
INSERT INTO SeldomUsedData (TransactionID, CategoryID, CustomerID, Quantity, UnitPrice, Discount)  
SELECT TransactionID, CategoryID, CustomerID, Quantity, UnitPrice, Discount FROM SalesTransactions;
```

Task 2:

Considering our dataset consists of 2 years of data, we can split it into partitions of 2, 1 for each year. In essence partition 1 is from 2020-2021, and partition 2 is from 2021-2022.

```
CREATE TABLE SalesTransactionsP1 (  
    TransactionID INT,  
    ProductID INT,  
    TransactionDate DATETIME,  
    Region VARCHAR,  
    TotalAmount FLOAT,  
    PRIMARY KEY (TransactionID)  
);
```

```

TransactionID INT,
ProductID INT,
CategoryID INT,
CustomerID INT,
TransactionDate DATETIME,
UnitPrice FLOAT,
Discount FLOAT,
TotalAmount FLOAT,
Region VARCHAR,
Quantity INT,
PRIMARY KEY (TransactionID),
CHECK (TransactionDate >= toDate(101-2020)AND TransactionDate <toDate(1-1-2021))
);

```

```

CREATE TABLE SalesTransactionsP2 (
TransactionID INT,
ProductID INT,
TransactionDate DATETIME,
CategoryID INT,
CustomerID INT,
Quantity INT,
UnitPrice FLOAT,
Discount FLOAT,
TotalAmount FLOAT,
Region VARCHAR,
PRIMARY KEY (TransactionID),
CHECK (TransactionDate >= toDate(1-1-2021) AND TransactionDate < toDate(1-1-2022))
);

```

Now we transfer the data,

```
INSERT INTO SalesTransactionsP1
```

```
SELECT *
```

```
FROM SalesTransactions WHERE TransactionDate >= toDate(1-1-2020) AND TransactionDate < toDate(1-1-2021);
```

```
INSERT INTO SalesTransactionsP2
```

```
SELECT *
```

```
FROM SalesTransactions WHERE TransactionDate >= toDate(101-2021) AND TransactionDate < toDate(1-1-2022);
```